Guest Languages
October 2026

joel.dice@akamai.com

Agenda

> Core Wasm and Component Model features of interest

> Status and TODOs for popular ecosystems

> Q&A

See also: Guest Languages February 2026

Core Wasm and Component Model Features of Interest

Core: Exception Handling and GC

> On by default as of Wasmtime 47 🎉
  > Thanks, F5!

> C++ exceptions supported as of WASI-SDK v33
  > and will work with shared libs and multithreading in v35

> Kotlin, Dart, Scala using GC+CM and eager for ABI support

Component Model: Cooperative Multithreading

> Experimental support in Wasmtime and Jco 🎉
  > Thanks, Fastly and Cosmonic!

> Experimental pthread support in WASI-SDK
  > Passes all applicable Python, Open POSIX, and Rust tests

> Plan to stabilize for WASI 0.3.3 or 0.3.4

> **Please help test with your real-world workloads!**

Component Model: async, streams, and futures

> WASI 0.3.0 shipped 🎉

> Fully supported in Wasmtime and Jco

> Enabled by default as of Wasmtime 46

> Native WASIp3 is feature-complete in WASI-SDK
  > Thanks, Alex and Christof!

> Still making occasional refinements as corner cases emerge
  > See Luke’s presentation for details

Component Model: Optional Imports

> No real updates, but it's still a priority!

> Needed to make adopting new WASI features in std libs practical

> Makes targeting specific worlds easier

  > No filesystem? No problem!

Ecosystem Details

Python Status

> Native WASIp3 support in `componentize-py` 0.26.0 (November)
  > including stdlib, multithreading, Cython (e.g. latest NumPy)

> WASI now a Tier 2 CPython platform
  > Thanks, Brett!
  > Will adopt WASIp3 upstream once 0.3.3 released

Python TODOs

> Still need solid Fortran->Wasm support for e.g. SciPy

> `ssl` module emulation using `wasi-tls`

> Unbundled components (e.g. reusable libpython3xx.so, etc.)

> WASI wheel tag PEP

> NightMonkey-style AoT?

Go Status

> Not much new to report; `componentize-go` chugging along
  > Thanks to Andrew Steurer for maintaining it!

> Several vibe-coded WASIp3 ports of the Go runtime to exist
  > They work great, apparently
  > Upstream issue stalled

Go TODOs

> Replace CPS transform with CM coop multithreading
    > Somebody please vibe this and post benchmarks!

> Convince upstream that WASIp3 support is worthwhile

JavaScript Status and TODOs

> See Till's presentation up next!

> Also, NightMonkey!

.NET Status and TODOs

> Currently stalled

> Scott Waye started async/p3 support, but needs more work

> No coop threads yet, but might “just work” via wasi-libc

> As with Go, need to convince upstream it's worth supporting

Rust Status and TODOs

> See Alex's presentation!

> `wasm32-wasip3` stabilizes in Rust v1.100 (early November)
    > multithreading still experimental; try it!

> `tokio` and `mio` WASIp2 support landed early this year
    > Bailey vibed p3 support with multithreading; looks promising

Others

> C: `wit-bindgen-c` and `wasi-libc` WASIp3 support is solid

> C++: `wit-bindgen-cpp` looking great; no async support yet

> Kotlin, Dart, and Scala: eager to push GC CM integration forward
    > Also curious about resource finalization and weak references

> Java, Swift, Ruby, etc.: ???

Q&A